Integrating Agentic AI into Software Development: A Sociotechnical Case Study in a Large Nordic Bank

Authors: **Daniel Blückert, Jacob Wahlström and Maximilian Wikström**

Semester 6

Informatics with a Focus on Information Systems, Bachelor's Course – IK300G

Subject: *Informatics*

Örebro University School of Business

Supervisor: Panagiota Chatzipetrou

Examiner:

Abstract

The field of software engineering is undergoing a fundamental transformation driven by the emergence of Agentic AI, autonomous systems capable of reasoning, planning and executing complex tasks with minimal human intervention. While these technologies promise significant efficiency gains, their integration into large and highly regulated environments such as the financial sector introduces sociotechnical challenges. The purpose of this study is to examine how a large Nordic bank can effectively integrate autonomous AI agents into its software development processes to enhance efficiency and scalability while maintaining security and quality. The research specifically explores the impact of workflows, team roles and decision making through Sociotechnical Systems (STS) theory and Human AI-Collaboration. This study follows a qualitative, where data was collected through semi-structured interviews with developers, architects and managers within the bank's IT organization. The data was analyzed using an thematic analysis. The findings reveal that agentic AI shifts the role of software developers from primary code authors to "Agent Coaches" focused on orchestration, context engineering and oversight. While efficiency increases for routine tasks, a significant trade-off emerges where significant human effort is required to provide structured business context and verify AI outputs. The study concludes that successful integration is not merely a technical deployment but a sociotechnical redesign requiring structured workflows, such as those proposed in the SASE framework, to maintain control and trust in regulated environments.

Keywords: Agentic AI, Sociotechnical Systems, Human AI-Collaboration, Financial Sector & Structured Agentic Software Engineering (SASE)

Table of Contents

1. Introduction 4
1.1 Background 5
1.2 Purpose and Research Question 6
1.2.1 Purpose. 6
1.2.2 Main research question. 6
1.3 Definitions 6
1.4 Scope and Delimitations 7
2. Theory/Previous Research 8
2.1 Sociotechnical Systems Theory 8
2.2 Human-AI Collaboration 9
2.3 Agentic Software Engineering 9
2.3.1 Autonomy Levels 9
2.3.2 Structured Agentic Software Engineering (SASE) 9
2.3.3 Empirical Insights from Early Deployments 11
2.4 Conceptual Distinctions Between AI Agents and Agentic AI 12
2.5 Implications and Risks of Agentic AI in Organizational Contexts 12
2.6 Autonomous Agents in Software Development 13
3. Method 15
3.1 Research Design: interpretative Instrumental Case Study 16
3.2 Inductive Research Approach 17
3.2 Data Collection Methods 17
3.2.1 Semi-Structured Interviews 17
3.2.2 Sampling Strategy 19
3.3 Data Analysis: Thematic Analysis 21
3.4 Research Quality and Trustworthiness 22
3.5 Ethical Considerations 24
3.6 Use of AI tools 25
4. Analysis and Results 25
4.1 Initial Adoption and Perceived Benefits 25
4.2 Evolving Developer Roles and the Need for Oversight 26
4.3 Technical limitations and challenges 28
4.4 Emerging Structures: Towards an Agent Execution Environment 28
4.5 Quality, Trust and Responsibility of AI-Generated Code 29
5. Discussion 31
5.1 Agentic AI as a source of efficiency 31
5.2 Shifting roles: from software developers to supervisors and mentors 31
5.3 Technical limitations as organizational challenges 32
5.4 The role of structure: SASE as a governance mechanism 32
5.5 Trust, quality, and accountability in regulated environments 33
5.6 Implications for integrating agentic AI at Bank© 33
6. Conclusion 34
6.1 Concluding findings 34
6.2 Contributions to research 35
6.3 Contributions to practice 36
6.4 Limitations and Suggestions for Future Research 37
References 39
Appendices 41
Appendix A: List of respondents 41
Appendix B: Interview Guide 42

1. Introduction

Agentic AI refers to a class of artificial intelligence systems designed to operate autonomously by interpreting high-level goals, planning multi-step actions, and executing tasks with limited or no continuous human intervention. Unlike traditional AI tools that react to user prompts or predefined inputs, agentic AI systems exhibit goal-directed behavior and can coordinate reasoning, tool use, and verification activities over extended workflows. In the context of software engineering, agentic AI enables autonomous agents to perform complex development tasks such as code generation, testing, refactoring, and decision-making, thereby shifting AI from a supportive role to an active participant in the development process (Hassan et al., 2025).

The theoretical foundation of this study is based on peer-reviewed research identified through academic databases such as Google Scholar and IEEE Xplore. Studies were selected based on relevance to autonomous software agents, agentic AI and sociotechnical theory.

The discipline of software engineering is currently undergoing a fundamental transformation driven by the maturation of artificial intelligence. While previous advancements in automation were deterministic, meaning they awaited explicit commands to execute predefined functions, the industry is now witnessing the rise of "Agentic AI." Unlike traditional tools, Agentic AI refers to autonomous systems capable of reasoning, planning, and acting to achieve complex goals with minimal human intervention (Pati, 2025).

This shift moves the field beyond simple code generation toward a new paradigm where AI systems function as autonomous teammates rather than passive instruments. Recent reviews indicate that these technologies are poised to redefine software development by automating complex tasks such as debugging, predictive maintenance, and decision-making (Alenezi & Akour, 2025). However, this transition introduces profound sociotechnical and regulatory challenges, particularly for organizations in high-stakes environments like the financial sector.

For large, regulated organizations such as Bank©, the adoption of agentic AI is particularly significant. The financial sector operates under strict requirements for transparency, auditability, and information security. Integrating AI agents into software development processes therefore demands new approaches to governance and ethical oversight.

Despite growing interest in AI-assisted programming, existing research primarily focuses on tools that augment human decision-making rather than agents that act autonomously. There is a lack of empirical studies exploring how agentic AI influences real-world development workflows, organizational structures, and team dynamics.

This thesis addresses that gap through a qualitative study conducted within Bank©'s IT organization. By examining several ongoing projects where agentic AI is being implemented, the study investigates how the introduction of autonomous agents affects workflows, roles, and decision-making. The research aims to generate both theoretical insights into sociotechnical change and practical knowledge about how organizations can integrate AI responsibly and effectively.

1.1 Background

Artificial intelligence (AI) has rapidly emerged as a transformative force in software engineering, reshaping how software is developed, tested and maintained (Alenezi & Akour, 2025). Recent advances in AI have led to the rise of agentic AI; autonomous AI systems capable of reasoning, planning, and acting with minimal human input.

The integration of autonomous agents into the development lifecycle has given rise to the concept of Agentic Software Engineering (SE 3.0). In this era, the role of the human developer shifts from being the primary author of code to becoming an supervisor of autonomous systems. Unlike traditional AI tools in software engineering, such as code completion plugins which are reactive and require a human to initiate every step, Agentic AI is characterized by proactivity and goal-directed behavior

As defined by Pati (2025), an agentic system can accept a high-level objective, such as "refactor this module to improve performance," and autonomously formulate a plan, execute necessary changes, and verify the results without continuous supervision. This capability represents a qualitative leap from "assisted" development to "delegated" development.

However, this development introduces both opportunities and challenges for software engineering organizations. Traditional methodologies such as Agile and DevOps rely heavily on human collaboration, iterative feedback and clearly defined team roles. The introduction of autonomous AI agents challenges these foundations by introducing non-human collaborators capable of making decisions and executing actions. This raises questions about accountability, division of labor, process control, and trust in AI-generated outputs.

For large, regulated organizations such as Bank©, the integration of agentic AI is particularly significant. The financial sector operates under strict requirements for transparency, auditability, and information security. Any adoption of AI agents must therefore align with governance frameworks, risk management protocols and ethical standards. This makes issues such as oversight, data protection, and responsible collaboration between humans and AI systems especially critical.

Despite the growing interest in agentic AI within software engineering research, existing studies are primarily conceptual and technical in nature, while empirical investigations of real-world organizational implementations remain scarce. Particularly limited is research that explores how such systems are integrated into established software development processes within large, highly regulated organizations. This represents a significant research gap, as organizational contexts characterized by strict security requirements, regulatory compliance and complex development environments pose unique challenges and constraints. Understanding how agentic AI can be effectively introduced under these conditions is therefore crucial, both from a practical and theoretical perspective.

1.2 Purpose and Research Question

1.2.1 Purpose.

The purpose of this study is to explore how agentic AI can be integrated into software development processes in large, regulated organizations in order to enhance efficiency and scalability while maintaining security and quality, from a sociotechnical perspective. By conducting an in-depth qualitative case study within a Nordic banking organization, the study aims to generate empirically grounded insights into both technical and organizational implications of agentic AI adoption.

By combining insights from Sociotechnical Systems Theory and Human–AI Collaboration research, the study seeks to deepen the understanding of both the technical and organizational implications of introducing autonomous agents into software development environments within regulated contexts. Particular attention is given to ethical and security considerations, including data protection, confidentiality, and responsibility for AI-generated code.

1.2.2 Main research question.

How can large, regulated organizations effectively integrate agentic AI into their software development processes to enhance efficiency and scalability while maintaining security and quality?

1.3 Definitions

To ensure a common understanding of the core concepts examined in this thesis, the following key terms are defined below.

Agentic AI: Agentic AI refers to autonomous artificial intelligence systems designed to pursue complex goals with minimal human intervention. Unlike traditional or generative AI which primarily responds to prompts to create content, Agentic AI possesses the capability to reason, plan, and execute multi-step workflows to achieve specific outcomes (Pati, 2025). These systems operate proactively rather than reactively and can manipulate external tools and environments to complete assigned tasks (Alenezi & Akour, 2025).

Efficiency: In this study, efficiency refers to reductions in development time, manual workload and cognitive effort, as well as improvements in workflow speed, coordination and task completion resulting from the integration of agentic AI into software.

Scalability: Scalability refers to the capacity of software development processes, tools and organizational structures to accommodate increasing workloads, system complexity and organizational growth without proportional increases in resource consumption or declines in performance.

Security: Security concerns the protection of sensitive data, system integrity, regulatory compliance and the prevention of vulnerabilities that may be introduced through AI-generated code, autonomous decision-making or automated system interactions.

Quality: Quality refers to the correctness, reliability, maintainability and readability of software code, as well as adherence to organizational standard, testing procedures and regulatory requirements.

Autonomous Coding Agents: These are specialized software agents capable of performing software engineering tasks such as code generation, debugging, testing, and refactoring without continuous human guidance. They utilize Large Language Models (LLMs) to understand high-level intent and autonomously navigate codebases to implement solutions (Yang et al., 2024).

Bank©: In the context of this study, a "Bank©" refers to a large-scale Nordic financial institution that is subject to strict regulatory frameworks and compliance requirements.

Large Language Models (LLMs): Deep learning models trained on vast datasets to understand, generate, and manipulate human language and code. In the context of this study, LLMs serve as the cognitive engine that powers autonomous agents, enabling them to interpret instructions and generate syntactic software solutions.

Sociotechnical Systems (STS): A sociotechnical system is an organizational work design that recognizes the complex interaction between social elements, such as people and team structures, and technical elements, such as software and infrastructure (Baxter & Sommerville, 2011). In this thesis, the introduction of Agentic AI is viewed as a modification of the existing sociotechnical system at Bank©, altering the relationship between human developers and their technical environment.

Structured Agentic Software Engineering (SASE): SASE is a theoretical framework proposed by Hassan et al. (2025) to standardize the collaboration between human developers and autonomous agents. It divides software engineering into two modalities: SE for Humans, where humans focus on intent and mentorship, and SE for Agents, where agents handle execution and verification through standardized protocols.

Human-in-the-Loop (HITL): A model of interaction where human oversight is required to validate, guide, or correct the actions of an autonomous system. In the context of Agentic AI, this refers to the necessity for developers to review agent-generated code and architectural decisions to ensure security and quality before deployment.

Context Engineering: The process of structuring business rules, architectural patterns, and requirements into machine-readable formats that AI agents can reliably interpret. Unlike simple 'prompt engineering,' context engineering involves curating the comprehensive environment and knowledge base required for an agent to operate autonomously within a specific domain.

Agent Execution Environment (AEE): A dedicated technical environment optimized for autonomous agents to execute tasks, run tests, and interact with tools. Distinct from a human Integrated Development Environment (IDE), an AEE is designed to facilitate high-velocity agent operations without the UI overhead required for human users.

Greenfield vs. Brownfield Development:

- **Greenfield Projects:** Software development projects lacking constraints from prior work. In these environments, agents can generate code without needing to navigate legacy technical debt.
- **Brownfield Projects:** Development and maintenance work performed on existing legacy software systems. These environments often contain undocumented logic and technical debt, presenting significant challenges for autonomous agents.

Human-AI Collaboration: Refers to the joint execution of tasks where humans and artificial intelligence systems work together as a team.

1.4 Scope and Delimitations

This study focuses on the integration of agentic AI within Bank© 's software development processes. The analysis includes organizational workflows, team roles, and collaborative practices affected by the introduction of autonomous agents. The study investigates how these agents are used in real development projects and how stakeholders perceive their impact.

The study does not examine:

- Internal AI model architectures or training methods
- Quantitative performance metrics of AI agents
- Technical benchmarking of AI accuracy or efficiency

These areas are outside the scope due to limited access to proprietary AI systems and because the study prioritizes organizational and process-oriented insights rather than AI engineering.

The participant scope comprises six key individuals within Bank© 's IT organization, identified through a combination of purposive and snowball sampling. To capture a sociotechnical view, the sample includes diverse roles: senior software developers, solution and domain architects, and team managers. Inclusion was strictly limited to those with direct experience using, piloting, or planning the implementation of autonomous AI systems.

Furthermore, the study is subject of certain methodological constraints regarding data collection. A portion of the interviews was conducted in English, which is a second language for both the interviewers and the respondents. This may have limited the linguistic nuance and depth of expression compared to native language communication. Additionally, technical challenges during the recording process, specifically microphone malfunctions and ambient noise in public settings resulted in partial audio loss for two interview sessions. In these instances, the analysis relies on field notes and immediate recollections rather than recordings. This is a limitation that should be considered when interpreting the findings from these specific interviews.

2. Theory/Previous Research

This chapter outlines the theoretical framework and previous research that form the foundation of this study. To analyze the impact of agentic AI on software development, the study primarily draws upon Sociotechnical Systems (STS) Theory, which provides a lens for understanding the interplay between technical tools and organizational social structures. Furthermore, research on Human-AI Collaboration is presented to explore the changing dynamics of responsibility and control in hybrid teams. Finally, the chapter reviews current literature on Agentic Software Engineering and relevant taxonomies to contextualize the technological shift from supportive tools to autonomous agents.

2.1 Sociotechnical Systems Theory

Sociotechnical Systems (STS) theory provides a theoretical foundation for this study. The theory posits that organizational systems consist of two interdependent components: the technical system, which includes tools, tasks, and technologies, and the social system, which encompasses roles, communication patterns, competencies, and cultural norms (Baxter & Sommerville, 2011). A core principle of STS is joint optimization, which argues that technical and social elements must be designed and adjusted in relation to one another to achieve effective and sustainable outcomes, rather than optimizing one at the expense of the other (Baxter & Sommerville, 2011).

Early sociotechnical research within the field of information systems further emphasized that system performance cannot be understood solely in technical terms. Boström & Heinen (1977) argue that information systems should be conceptualized as sociotechnical systems in which technical components, work tasks, and social structures are tightly interrelated. Changes in one subsystem influence patterns of coordination, task execution, and distributions of responsibility, while existing organizational arrangements simultaneously shape how technological systems are implemented and used.

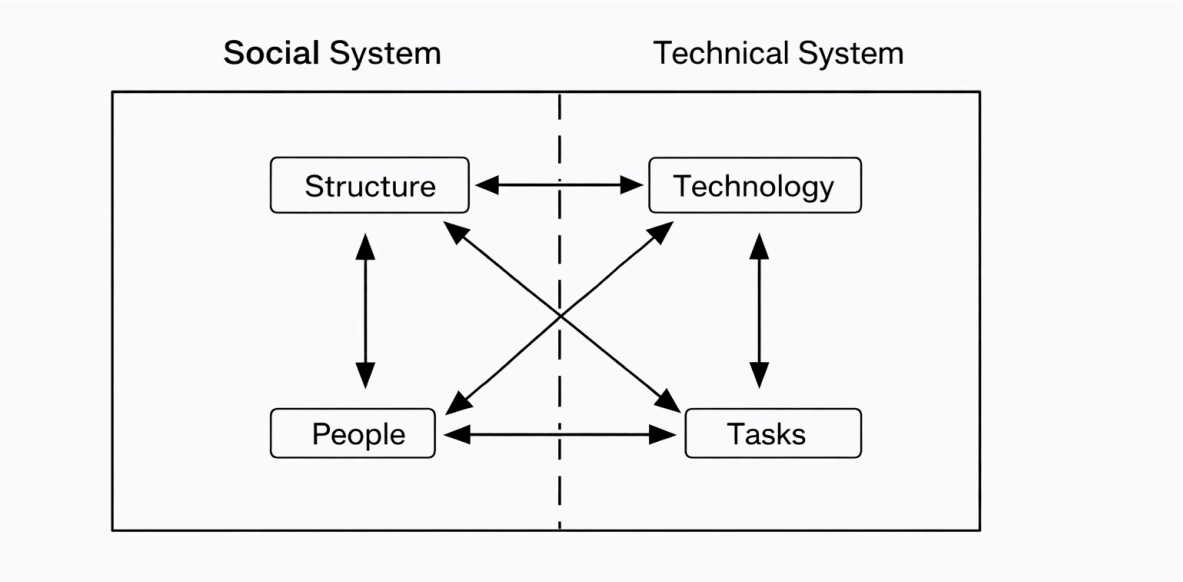

Figure 1. *Sociotechnical subsystems and joint optimization* (Bostrom & Heinen, 1977).

As shown in Figure 1, the technical system, social system, task structures, and organizational arrangements are mutually influencing components. The model highlights that organizational performance depends on the alignment between these interconnected subsystems rather than on technical optimization alone. A change in the technical subsystem, such as the introduction of a new information system, will therefore require corresponding adjustments in work practices, governance mechanisms, and responsibility structures within the social subsystem.

A key concept emerging from this perspective is misalignment. When technological capabilities evolve without corresponding adaptations in roles, routines, competencies, or accountability structures, system effectiveness may be undermined despite technically successful implementation (Boström & Heinen, 1977). Misalignment can manifest as increased coordination costs, unclear responsibility boundaries, reduced trust in system

outputs, or resistance among users. STS therefore provides a framework for analyzing not only successful integration but also the organizational tensions that may arise during technological change.

In the context of emerging autonomous technologies, STS offers a systematic lens for understanding how the introduction of agentic AI reshapes work structures, decision processes, and role distributions. The integration of advanced automation requires not only technical implementation but also corresponding shifts in human responsibilities, oversight mechanisms, and organizational routines (Baxter & Sommerville, 2011). As autonomous agents assume tasks traditionally performed by human developers, responsibilities related to supervision, validation, and accountability may be redistributed within the organization.

Consequently, STS is central to analyzing how software engineering environments must adapt to maintain coherence, accountability, and quality when introducing autonomous agents. From an STS perspective, successful integration depends on maintaining alignment between the technical subsystem and the social and organizational arrangements within which it operates, particularly in highly regulated environments.

2.2 Human-AI Collaboration

As a complement to STS, research in Human-AI Collaboration provides insights into how humans and intelligent systems jointly contribute to tasks and decision-making. This field emphasizes that AI systems function as collaborators rather than isolated tools, with humans remaining responsible for supervision, ethical judgment and strategic direction (Schneiderman, 2020). Concepts such as human-in-the-loop and human oversight are central themes in this literature, advocating for design strategies that ensure high levels of human control even as automation capabilities increase (Schneiderman, 2020).

Effective interaction requires clear communication structures, well-defined roles, and mechanisms that allow humans to monitor, guide and intervene in AI agents' activities when necessary (Schneiderman, 2020). This theoretical perspective is valuable because it highlights the cognitive, organizational and procedural conditions needed for productive cooperation between humans and autonomous AI systems. It also complements STS by focusing specifically on the dynamics of hybrid human-machine teams, which is highly relevant to the emerging integration of agentic AI (Dellermann et al., 2019).

2.3 Agentic Software Engineering

Recent advancements in artificial intelligence have led to the development of agentic AI systems capable of conducting complex, multi step software engineering tasks. The research field of Agentic Software Engineering explores how such autonomous agents can be systematically integrated into software development workflows.

A central contribution in this area is the study by Hassan et al. (2025) Agentic Software Engineering: Foundational Pillars and a Research Roadmap, which outlines conceptual models, autonomy levels and structured collaboration frameworks for agentic AI systems.

2.3.1 Autonomy Levels

Hassan et al. (2025) propose a hierarchy of autonomy levels for AI in software engineering, ranging from fully manual human coding at Level 0 to fully autonomous, general purpose

AI development at Level 5. Of particular relevance is Level 3, which describes goal agentic automation. At this level, an AI agent can interpret high level objectives, plan multi step actions, generate code and perform testing with limited human intervention. This stage represents a significant shift from predictive text or token based code completion toward deliberate, structured automation that performs larger software development activities.

Levels 4 and 5 describe future scenarios in which AI systems possess domain specific or fully general autonomy across software engineering tasks. These levels remain theoretical but indicate the direction in which agentic AI research is progressing. The autonomy hierarchy provides a conceptual map of how AI capabilities evolve and where current technologies are positioned relative to fully autonomous development.

2.3.2 Structured Agentic Software Engineering (SASE)

Hassan et al. (2025) also introduce the Structured Agentic Software Engineering (SASE) framework, which outlines how humans and AI agents can collaborate within organized and transparent workflows. SASE defines two working modalities: SE for Humans (SE4H), where humans provide strategic direction and oversight, and SE for Agents (SE4A), where autonomous agents carry out execution oriented tasks.

The framework identifies four foundational pillars that require reconfiguration in the agentic era:

1. **Actors**: New hybrid roles emerge, including agents as software contributors and humans as supervisors or mentors.
2. **Processes**: Human–agent interactions must be standardized to ensure predictability and traceability.
3. **Artifacts**: Collaboration relies on structured artifacts such as Briefing Packs, Consultation Requests and Merge Readiness Packs.
4. **Tools**: Dedicated environments are needed, including an Agent Command Environment (ACE) for humans and an Agent Execution Environment (AEE) for agents.

The SASE framework formalizes how humans and autonomous agents collaborate across structured workflows. It divides the engineering process into dual operational domains and introduces artifacts such as the Briefing Pack, Consultation Request Pack and Merge Readiness Pack that regulate information exchange (Hassan et al., 2025, Fig. 3).

Beyond defining roles and workflows, the SASE framework can be understood as a governance-oriented response to the increasing autonomy of AI agents in software engineering. Hassan et al. (2025) emphasize that as development activities shift from human-authored code toward delegated, goal-oriented agent execution, traditional informal coordination mechanisms become insufficient. SASE therefore introduces structured, durable, and suitable artifacts as first-class elements of the engineering process, replacing ad-hoc prompting and transient conversational interactions with formalized information exchange (Hassan et al., 2025). This shift reflects a broader movement from craft-based practices toward disciplined engineering suitable for large-scale and regulated environments.

A central contribution of SASE lies in its treatment of artifacts as interfaces between humans and agents rather than mere documentation. Artifacts such as Briefing Packs,

Consultation Request Packs, and Merge Readiness Packs function as contractual boundary objects that encode intent, constraints, verification evidence and accountability in a machine-readable yet human-interpretable form (Hassan et al., 2025). This design directly addresses concerns related to traceability, reproducibility and auditability, which are particularly critical in high-regulation organizational contexts. By externalizing decision rationales and verification evidence into structured artifacts, SASE reduces reliance on opaque agent reasoning traces and enables systematic human oversight.

From a sociotechnical perspective, SASE operationalizes the principle of joint optimization by explicitly redesigning both social roles and technical infrastructures in tandem. Human developers transition into roles resembling supervisors, mentors or "agent coaches", while AI agents assume responsibility for execution-intensive tasks with clearly bounded operational environments (Hassan et al., 2025). This redistribution of labour aligns with Sociotechnical Systems Theory by recognizing that technological autonomy necessitates corresponding changes in organizational roles, processes and accountability structures. Rather than treating AI agents as isolated tools, SASE embeds them as actors within a coordinated sociotechnical system.

Furthermore, the dual-environment model consisting of the Agent Command Environment (ACE) and the Agent Execution Environment (AEE) reinforces separation of concerns between strategic control and operational execution. ACE supports human cognition by prioritizing observability, review and decision-making, while AEE is optimized for agent autonomy, parallelism and high-velocity execution (Hassan et al., 2025). This separation mitigates the risk of cognitive overload for human developers and addresses the "speed versus trust" tension observed in early deployments of autonomous coding agents. In doing so, SASE provides a structural mechanism for scaling agentic productivity without eroding quality assurance practices.

Overall, SASE contributes a theoretically grounded and practically oriented framework for governing human-agent collaboration in software engineering. By formalizing interaction patterns, redefining roles and introducing structured artifacts as coordination mechanisms, the framework complements existing research on human-AI collaboration and extends STS theory into the domain of agentic software engineering. As such, SASE offers a valuable conceptual lens for analyzing how organizations can integrate autonomous AI agents while maintaining control, accountability and trust in complex development environments.

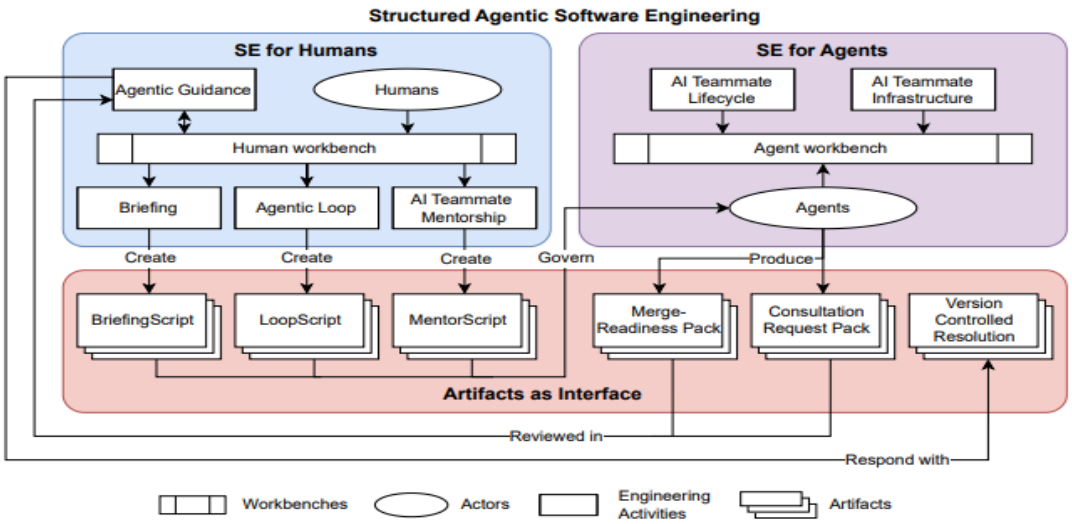

Figure 2. *Foundational pillars of Agentic Software Engineering* (Hassan et al., 2025).

2.3.3 Empirical Insights from Early Deployments

Hassan et al. (2025) also present empirical observations from real world applications of agentic AI systems. These observations indicate that agent generated code can significantly accelerate development workflows, but they also highlight challenges such as increased pressure on human code review capacity, variability in solution quality and the need for robust verification processes.

The findings illustrate that while agentic AI has the potential to enhance productivity, it also introduces new organizational and quality assurance challenges that require structured integration and oversight. These insights enrich the theoretical understanding of how agentic AI operates in practice.

2.4 Conceptual Distinctions Between AI Agents and Agentic AI

A complementary perspective is provided by the study AI Agents vs. Agentic AI: A Conceptual Taxonomy, Applications and Challenges by Sapkota et al. (2026). The authors develop a taxonomy that differentiates between single AI Agents and multi agent, orchestrated Agentic AI systems. According to Sapkota et al. (2026), AI Agents operate as single systems performing task specific activities using reasoning, tool use and limited memory. They are suitable for narrow, well defined tasks.

In contrast, Agentic AI systems consist of multiple interacting agents that can collaborate, reflect and restructure tasks dynamically. These systems integrate planning mechanisms, hierarchical coordination, long term memory modules and orchestration components that enable distributed problem solving and adaptive behavior.

Sapkota et al. (2026) further identify challenges associated with both approaches, ranging from hallucinations and reasoning limitations in single agents to coordination failures and emergent errors in multi agent systems. Their taxonomy broadens the conceptual foundation of autonomous AI and complements frameworks such as the SASE model by

emphasizing the architectural and systemic properties that influence how autonomous AI behaves in complex environments.

2.5 Implications and Risks of Agentic AI in Organizational Contexts

An additional perspective on agentic AI is provided by Murugesan (2025) in The Rise of Agentic AI: Implications, Concerns, and the Path Forward. This study offers a broad, cross-industry analysis of the opportunities, risks and systemic challenges associated with the deployment of agentic AI systems. While prior research primarily focussed on technical architectures and software engineering workflows, Murugesan (2025) emphasizes the wider organizational, ethical and societal implications that accompany the integration of autonomous agents.

According to Murugesan (2025), the emergence of agentic AI is accelerating due to advances in multimodal models, reasoning capabilities and autonomous planning. The article highlights that organizations across sectors are increasingly adopting agentic AI for tasks such as workflow automation, knowledge management and operational decision support. Industry forecasts cited in the report indicate substantial growth in enterprise adoption, with expectations that agentic AI will become a key component of future digital transformation initiatives.

A major contribution of this study is its systematic categorization of the risks and challenges associated with agentic AI. Murugesan (2025) identifies several categories of concern, including safety risks, privacy and data protection issues, algorithmic opacity, misuse, system failures and emergent harmful behavior. The report notes that as autonomous agents gain the ability to make complex decisions, manage workflows and act without continuous human supervision, the potential consequences of malfunction or unintended outcomes increase.

The study also highlights governance and accountability challenges. Agentic systems may operate through opaque reasoning processes, making it difficult for organizations to audit decisions, trace errors or assign responsibility for outcomes. Murugesan (2025) stresses the need for robust governance models, ethical guidelines and oversight mechanisms that ensure transparency, safety and human alignment throughout the lifecycle of agentic AI deployment. The article argues that without such structures, organizations risk systemic vulnerabilities, regulatory non-compliance and erosion of stakeholder trust.

Another important theme in the study is organizational transformation. Murugesan (2025) describes how agentic AI reshapes work roles, requiring new skills, updated processes and cross-functional coordination. Human workers are expected to shift toward supervisory, interpretive and oversight-oriented responsibilities as autonomous agents take on routine or complex operational tasks. This aligns with broader discussions in STS theory regarding the co-evolution of technology and social structures.

Finally, Murugesan (2025) proposes a path forward centered on responsible innovation. This includes integrating human oversight into autonomous workflows, establishing risk mitigation frameworks and implementing continuous monitoring to detect and manage unexpected agent behavior. These recommendations underscore the importance of embedding agentic AI within structured organizational systems that balance autonomy with safety and accountability.

Overall, Murugesan's (2025) study contributes to a broader understanding of agentic AI by illuminating the ethical, governance and organizational dimensions of autonomous systems. It complements technical frameworks such as SASE and conceptual taxonomies of agentic AI by demonstrating that successful integration requires not only advanced algorithms but also robust oversight structures, organizational adaptation and responsible management practices. This perspective strengthens the theoretical foundation for examining agentic AI within complex socio-technical environments.

2.6 Autonomous Agents in Software Development

A further contribution to the understanding of autonomous AI in software engineering is the vision paper Autonomous Agents in Software Development: A Vision Paper by Rasheed et al. (2018). Although framed as a forward-looking exploration rather than an empirical study, the paper offers valuable conceptual insights into how autonomous agents may reshape key activities within the software development lifecycle. Vision papers such as this play an established role in emerging fields by outlining plausible future trajectories, identifying potential roles for new technologies and highlighting challenges that require further scientific investigation.

Rasheed et al. (2018) propose that future autonomous agents could take on responsibilities across several phases of software development, including code generation, code review, debugging, maintenance and documentation. The authors illustrate scenarios in which autonomous agents collaborate to solve software engineering tasks, each with specialized capabilities and coordinated through multi-agent reasoning frameworks . These scenarios align with broader research trends indicating that autonomous systems may increasingly assume responsibilities beyond simple predictive coding assistance.

The study also discusses the potential benefits of such agents, including increased development efficiency, reduction of repetitive tasks and enhanced consistency in code quality. However, Rasheed et al. (2018) emphasize that realizing these benefits requires overcoming several limitations associated with large language model based agents. These include challenges related to incomplete context understanding, limited memory capacity, susceptibility to hallucinations and difficulties in validating generated code or ensuring alignment with system requirements . The paper therefore highlights the importance of structured oversight, complementary human involvement and improvements in agent collaboration protocols.

Another notable contribution of the paper is its reflection on how autonomous agents may impact existing software engineering roles. Rasheed et al. (2018) argue that traditional roles within development teams have evolved according to human strengths and constraints, such as limitations in memory, processing speed and error proneness. Autonomous agents, by contrast, may not share these characteristics, which implies that future software engineering practices may need to be restructured to integrate these systems effectively and safely. This perspective resonates with principles from Sociotechnical Systems Theory, which emphasizes the need for organizational structures and technical systems to evolve in tandem.

While the paper does not present empirical evaluations, it provides a conceptual starting point for understanding how autonomous agents might operate collaboratively within complex software engineering environments. It also identifies several open research

questions relating to coordination mechanisms, verification processes and the design of mixed-initiative workflows. These areas remain central to current research and complement the structured frameworks and taxonomies presented in recent literature on agentic AI.

Overall, the vision outlined by Rasheed et al. (2018) enriches the theoretical foundation of autonomous AI in software development by presenting potential future applications, challenges and reconfigurations of work practices. As a conceptual perspective, it complements the empirical and architectural studies discussed earlier and offers a forward-looking lens on the evolving role of autonomous agents in the software engineering landscape.

3. Method

This chapter details the research design, data collection and analysis methods used to investigate how agentic AI can be integrated into software development processes in large, regulated organizations in order to enhance efficiency and scalability while maintaining security and quality. It also addresses the measures taken to ensure research quality and ethical considerations.

Table 1: Alignment of Methodological Choices with Research Goals

Methodological Choice	**Description**	**Alignment with Research Goals**
Qualitative Strategy	Focus on words, meanings, and an inductive approach to qualitative analysis (allowing themes to emerge from the data without predefined analytical categories) (Bryman, 2016).	Allows in-depth understanding of a novel, complex phenomenon (agentic AI integration) through rich, nuanced data rather than superficial metrics.
Epistemology: Interpretivism	Understanding social action through participants' perspectives (Bryman, 2016). Reality is viewed as socially constructed by human meaning-making rather than an objective fact.	Focuses on how developers, architects, and other stakeholders interpret and make sense of AI agents in their work; capturing their subjective experiences and sensemaking processes.
Ontology: Constructionism	Social properties are outcomes of interactions, not fixed entities (Bryman, 2016). Concepts like "accountability" or "code quality" are seen as fluid constructs shaped by sociotechnical interactions.	Highlights that organizational realities (e.g. definitions of quality, responsibility) are being re-negotiated through the introduction of autonomous AI agents. This view helps the research capture how human-AI interaction at Bank© **reconstructs** these concepts over time.
Research Design: Single Case Study	In-depth, intensive analysis of a single "critical case," emphasizing context and complexity over	Bank© is treated as a critical case; a highly regulated bank environment presenting unique challenges for

	breadth (Bryman, 2016). Provides a **"thick description"** of processes within one setting	agentic AI integration. Focusing on one case allows rich contextual insight into integration processes under real-world constraints, rather than superficial comparison across sites. Insights gained aim to be theoretically transferable to similar organizations (instrumental case rationale).

3.1 Research Design: interpretative Instrumental Case Study

This study employs a qualitative, interpretive case study design. A qualitative approach is appropriate because the research seeks to understand how and why a sociotechnical change is occurring, questions that demand rich, explanatory data rather than numerical measurement. According to Bryman (2016), qualitative research prioritizes words and meanings over quantification and is well suited to inductive approaches where analytical insights are derived from empirical material rather than predefined theoretical categories. In this context, a quantitative survey would be insufficient, as it would impose predefined variables on a phenomenon that is still emerging and not yet well-defined. By contrast, a qualitative design allows the exploration of developers' and architects' subjective experiences and interpretations of working with autonomous AI agents, capturing nuances that a fixed-response might miss.

Epistemologically, the study aligns with interpretivism, which holds that social reality is understood by grasping the meanings that people assign to their experiences (Bryman, 2016). Rather than seeking an objective truth about AI tools, the research recognizes that concepts such as an AI agent being a "useful collaborator" or a "security risk" are not inherent properties of the technology but are socially constructed through discourse, regulation, and individual perceptions. This stance allows the research to capture how Bank© stakeholders themselves interpret agentic AI and the changes it brings to their work environment. Ontologically, a constructionist view is adopted: organizational realities (like "accountability" or "code quality") are not fixed, but are continually co-created through human–technology interactions. As agentic AI systems begin to generate code and make decisions, they actively reshape norms and practices; a constructionist lens sensitizes the researcher to observe how such norms are negotiated and redefined in real time through these new human-AI interactions.

Within this paradigm, the research design is a single-case study focusing on Bank© 's software engineering context. A case study involves detailed, intensive analysis of one bounded "case," and is well-suited for examining complex phenomena within their real-life context (Bryman, 2016). Bank© was chosen as a "critical case" (a particularly illuminating example) of a large, regulated financial institution confronting the challenges of integrating autonomous AI into development practices. The financial sector's strict regulatory requirements (e.g. GDPR, DORA) and low tolerance for error make Bank© an information-rich case to observe sociotechnical tensions that might remain less visible in more permissive environments. Studying a single case enables a thick description of the integration process. Capturing contextual details such as Bank©'s organizational culture, legacy systems, and governance structures that shape how agentic AI is adopted. A

cross-company comparison or broad survey, by contrast, would likely strip away context and yield only superficial data on "adoption rates," without explaining the how/why processes underlying integration. As Bryman (2016) notes, case studies are advantageous when the aim is to understand the contextual complexity of the processes in question.

While the primary purpose of the study is to examine how Bank© can integrate agentic AI into its software development processes, the findings also provide empirical insights that may be relevant to ongoing discussions on human–AI collaboration and sociotechnical systems in high-regulation contexts.

3.2 Inductive Research Approach

This study follows an inductive research approach, where analytical insights are derived from the empirical material rather than imposed by pre-existing theoretical frameworks. An inductive approach is appropriate when studying emerging and insufficiently theorized phenomena, as it allows patterns, meanings, and themes to develop directly from participants' accounts (Bryman, 2016).

In line with this approach, no theory was used to guide the coding process or the generation of themes. Instead, the analysis was grounded in the interview data, allowing participants' experiences and interpretations of agentic AI integration to shape the analytical outcomes.

Theoretical frameworks, including Sociotechnical Systems (STS) theory and Human–AI Collaboration, were applied after the inductive thematic analysis as interpretive and reflective lenses. Their role was not to structure the analysis, but to support the interpretation and discussion of the empirically derived themes by situating the findings in relation to existing research and theoretical concepts.

This separation between inductive analysis and theory-informed discussion ensures methodological consistency between the research approach, the analytical procedure, and the presentation of results.

3.2 Data Collection Methods

3.2.1 Semi-Structured Interviews

The primary data collection method was semi-structured interviews with key participants involved in Bank© 's agentic AI initiatives. Semi-structured interviews were chosen for their balance of focus and flexibility: we worked from an interview guide (Appendix B) covering specific topics, but the conversation can flow freely and probe new insights that arise (Bryman, 2016). This format is well-suited for "how" and "why" questions and was directly motivated by the study's aims. Exploring how different roles experience the shift to autonomous AI tools requires depth and openness in questioning that quantitative methods (e.g. surveys) cannot provide. As Bryman (2016) observes, a semi-structured interview is ideal when the researcher has defined themes to inquire about, yet wants to give interviewees substantial leeway in how they respond.

In the context of agentic AI, a novel and rapidly evolving phenomenon, this flexibility was crucial. A rigidly structured questionnaire might miss emerging issues that we did not anticipate in advance. By contrast, the semi-structured approach allowed the interviewer to probe unexpected answers and follow up on interesting points. For instance, if a developer mentioned encountering "prompt engineering fatigue" or a creative workaround for an AI

tool's limitation, the interviewer could deviate from the prepared questions to explore these insights in depth. This proved valuable for capturing cutting-edge, ground-level challenges and innovations that were not yet documented elsewhere.

Secondly, semi-structured interviews prioritize the "insider perspective" (Bryman, 2016). They enable participants to describe experiences in their own terms, yielding a richer understanding of their feelings and perceptions. In this study, it was important to hear directly from developers and architects about their lived experiences. Whether it be the frustration of debugging AI-generated code, or the satisfaction of accelerated development cycles. These nuanced emotional and professional insights can only surface through an open dialogue. By encouraging interviewees to share stories and examples (rather than just yes/no answers), the method captured the complex human side of introducing AI agents into software teams.

Thirdly, the semi-structured format offered contextual sensitivity: the interviewer could tailor questions to each participant's role and expertise. For example, with a Domain Architect the conversation might naturally gravitate towards topics like system governance, overall architecture, and compliance implications, whereas a Senior Developer's interview would focus more on day-to-day workflow changes, coding practices, and tool integration. The ability to adjust on the fly, reordering questions, rephrasing them in familiar terms, or probing specific topics, helped ensure that each interview was relevant and credible. It also enhanced validity by making sure participants could speak to what they knew best, reducing the chance of misunderstanding or superficial answers. All interviews were conducted as relaxed conversations with a purpose, creating rapport and trust so that participants felt comfortable discussing sensitive issues (Burgess, 1984, as cited in Bryman, 2016). This was particularly important given that topics like security risks, compliance, or role changes can be delicate; a trusting interview atmosphere encouraged honesty and fuller disclosure.

Interview Guide Development: An interview guide (Appendix B) was developed to ensure coverage of key topics while allowing flexibility. The guide included open-ended questions on themes such as: the participant's role and background; their experience with agentic AI tools; changes in workflow and collaboration since introducing AI; perceptions of the AI agent's autonomy and reliability; issues of responsibility, oversight, and accountability; technical challenges or enablers for integration; and any ethical, security, or compliance considerations noted in practice. These topics were derived from the research questions and literature on human-AI collaboration. The wording and sequence of questions were not fixed, but the guide ensured each broad area was explored with all participants for comparability. Additionally, the guide incorporated prompts and follow-ups, for example, asking for specific examples ("Can you describe a recent incident where the AI made an unexpected decision?"), to elicit concrete details rather than abstract statements. Bryman (2016) notes that using prompts can deepen the inquiry and keep the conversation flowing. Where appropriate, a critical incident technique was used: participants were invited to recall specific events (e.g., a notable success or failure involving the AI agent) to ground their opinions in real experiences.

Before formal interviews commenced, the guide was piloted and refined. An initial draft of questions was reviewed with the thesis supervisor and tested in a preliminary discussion with a colleague to ensure clarity and relevance. Early versions of the guide were found to focus too broadly on generic "AI tools" (such as ChatGPT-like applications) rather than the

specific concept of agentic AI being studied. Based on feedback from these pilot runs and additional context-gathering at Bank©, the guide was revised to target autonomous AI agents more explicitly. Using terminology familiar to participants (e.g., referring to the internal project name "Cloud Saga" when appropriate). Questions that were too technical or outside the scope (for instance, delving into AI model architecture details) were removed in favor of questions about process, experiences, and perceptions, aligning the interview focus with the study's sociotechnical perspective. This iterative refinement process, including the use of a pilot interview, helped improve the guide's clarity and ensured that each question was meaningful to participants. It mirrors Bryman's (2016) recommendation to conduct pilot interviews to refine research instruments for qualitative inquiries, thereby enhancing the reliability of data collection through careful design and testing.

Interview Procedure: A total of 6–8 interviews were planned (and ultimately conducted within this range). Participants were drawn from various roles to capture multiple perspectives on the AI integration. The sample included senior software developers (providing insight into day-to-day coding and tool use), solution or domain architects (offering a higher-level view of system design, strategy, and governance issues), and engineering managers or team leads (addressing management and process change perspectives). Each interview was scheduled for approximately 45–60 minutes and conducted one-on-one (either in person or via a secure video-call platform). Interviews were audio-recorded to ensure an accurate record, and later transcribed verbatim for analysis. Recording and transcription preserve participants' exact words and nuances, which is vital for reliable qualitative analysis (Bryman, 2016 emphasizes the importance of careful transcription in qualitative research). During the interview, we also took brief field notes to mark any non-verbal cues or immediate reflections. At the conclusion of each interview, participants were thanked and given an opportunity to add further thoughts or ask questions about the study. Overall, the semi-structured interviews generated rich narrative data on how agentic AI tools are perceived and utilized across different layers of Bank© 's IT organization.

3.2.2 Sampling Strategy

The study employed a non-probability sampling approach, specifically a combination of purposive sampling and snowball sampling, to identify participants who could provide rich and relevant insights. In qualitative research, representativeness in a statistical sense is not the goal; instead, the aim is to select participants who are information-rich with respect to the research questions (Bryman, 2016). Given that agentic AI use was (at the time of study) limited to certain pockets within Bank© 's IT organization, a random sample of employees would likely include many who have no exposure to the phenomenon, thus diluting the data. Therefore, purposive sampling was the primary strategy: participants were deliberately selected based on predefined criteria ensuring their relevance to the topic (Bryman, 2016 defines purposive sampling as selecting cases based on their direct connection to the research questions).

Inclusion Criteria: We established clear criteria to guide selection. Participants had to: (1) be currently employed in Bank©'s IT division (to understand the internal context), (2) hold a role directly related to software development, architecture, or a closely related technical leadership, and (3) have experience with agentic AI tools, either actively using an autonomous AI system, involved in piloting such a tool, or responsible for planning its implementation. In practice, this meant targeting individuals known to be working on

projects involving AI-driven coding assistants, autonomous testing bots, or similar initiatives (for example, those involved in the "Cloud Saga" proof-of-concept, or leading experiments with LLM-based coding tools). By focusing on those "in the center of the action", the sampling ensured that each interviewee could provide substantive, firsthand insight into the integration of agentic AI, rather than speculative opinions. This selection is sometimes called criterion sampling, a subtype of purposive sampling where all participants meet key conditions relevant to the study.

The initial pool of candidates was identified through discussions with our liaison in the Bank© 's IT department and by reviewing internal communications to see who was publicly associated with AI initiatives. We also considered role diversity to capture multiple perspectives: ensuring a mix of developers, architects, and managers.

Snowball Sampling: In addition to purposive selection, snowball sampling was utilized to expand the participant pool. Snowball sampling involves asking early informants to recommend other people who fit the study criteria (Bryman, 2016). This technique was valuable in the Bank© context because some of the most relevant actors in AI adoption might not be immediately obvious from organizational charts or official job titles. After conducting the first few interviews, we would ask each participant a question along the lines of: "Who else do you think I should talk to about agentic AI here at Bank©?" or more specifically, "Is there someone deeply involved in this project that you believe has important insights?" This led to additional participants, such as a Domain Architect who was referred by a Solution Architect as being pivotal in the AI proof-of-concept phase. Snowball referrals helped uncover "hidden populations", individuals who were heavily engaged with the AI tools but might not have been identified initially, such as a senior developer in another department experimenting with an autonomous testing script, or a compliance officer who had strong views on AI use.

One challenge of snowball sampling is the potential for bias: respondents might nominate colleagues who share similar perspectives or within their close network, which can lead to homogenous viewpoints (Bryman, 2016). We mitigated this by actively seeking diverse referrals. Participants were explicitly encouraged to point us not only toward other enthusiasts, but also toward those who might be critical or have different vantage points ("e.g., is there anyone who has raised concerns or skepticism about this AI initiative?"). This approach led us to include a participant from the governance/risk side who was more cautious about AI, balancing the sample. By tracing social networks in multiple directions, we captured a more varied set of perspectives.

Sample Size and Saturation: Ultimately, seven conducted. In qualitative research, sample adequacy is evaluated by the concept of theoretical saturation rather than sheer size: essentially, enough interviews are done when additional interviews no longer provide new insights (Bryman, 2016). Given the specificity and expertise level of our participants, even a relatively small number provided rich data. We found that by around the 6th interview, major themes were repeating and no fundamentally new themes were emerging, an indication of reaching saturation for our focused research question. Warren (2002) suggests that for intensive interview studies on a clearly defined phenomenon, a smaller number of high-quality interviews can be sufficient, especially if participants are experts or deeply involved in the issue. Each interview transcript was detailed and dense with information, since participants had significant experience and reflective insight on agentic AI in their workflows. The final set of ~6 interviews struck a balance between depth and coverage: it

was enough to capture a range of perspectives and establish confidence in the consistency of key findings, while allowing us to thoroughly analyze each transcript in detail given the time and scope.

3.3 Data Analysis: Thematic Analysis

The data collected through the interviews were analyzed using thematic analysis following Braun and Clarke's (2006) six-step framework. This framework offers a systematic and flexible process for identifying patterns within qualitative data. This method is particularly well suited for studies seeking to explore experiences and perceptions without binding them to a predetermined theoretical structure. With the exploratory nature of this thesis in mind this method was considered an appropriate analytical strategy.

The first step of this process consisted of us familiarizing ourselves with the material by listening to the recordings of the interviews while reading the transcripts multiple times while noting preliminary observations and analytical ideas. In the second step initial codes were generated through an inductive and semantic process. This means that the codes were derived directly from the data rather than guided by predefined theoretical concepts (Bryman, 2018). Coding was conducted manually, allowing us to get a deeper understanding of the material and continuously reflect over emerging meanings and themes throughout the process.

The third step involved organizing and grouping the generated codes into potential themes. This was done by identifying recurring patterns and meaningful relationships across the data. During the fourth step these themes were reviewed in relation to both the coded extracts and the full dataset. This iterative process ensured internal consistency within the themes as well as a clear distinction between them (Nowell et al., 2017). Themes that lacked sufficient empirical support or overlapped conceptually were revised, merged or excluded.

In the fifth step the final themes were named and defined. This step focused on articulating the analytical importance of each theme and clarifying how it contributes to the overall understanding of the thesis research questions (Braun & Clarke, 2006). Finally in the sixth step the themes were written up and are presented in the Result- chapter. To strengthen transparency and demonstrate how the analysis is grounded in the empirical material, illustrative and suitable quotes from the participants are included in the presentation of the results.

Throughout the analysis, we held ongoing discussions about our interpretations, which contributed to reflexivity and agreement amongst us. Such collaborative reflection has been highlighted as an important marker of quality in qualitative research (Tracy, 2010). The analysis was iterative, and the themes developed progressively as our understanding of the data deepened.

3.4 Research Quality and Trustworthiness

In qualitative research, the conventional criteria of validity and reliability are reinterpreted through the concept of trustworthiness (Bryman, 2016). Trustworthiness encompasses four aspects, credibility, transferability, dependability, and confirmability, which roughly parallel internal validity, external validity, reliability, and objectivity in quantitative

research. This study took several measures to maximize trustworthiness, ensuring that our findings are robust and believable as a qualitative account.

- **Credibility:** Credibility refers to the confidence that the study's findings accurately reflect the realities of the participants. We bolstered credibility through triangulation and member validation. Triangulation was achieved by cross-verifying information from multiple sources. Insights from interviews with one role (e.g., architects) were compared with those from another role (e.g., developers or managers) to see if they converged or offered complementary perspectives. We also practiced a form of member (respondent) validation during interviews: we would occasionally summarize or paraphrase a key point back to the interviewee and ask if that interpretation was correct. This check ensures we are not misinterpreting their statements. In a few cases, follow-up emails were sent to participants to clarify unclear parts from the interview; this feedback loop further enhances credibility by aligning our recorded data with participants' intended meanings. Additionally, the prolonged engagement with the case contributed to credibility: spending substantial time in the research context allowed us to understand nuances and check for misinformation or inconsistencies over time.

- **Transferability:** Transferability is concerned with the applicability of findings to other contexts. In qualitative work, the researcher does not claim that the findings are universally generalizable; instead, the aim is to provide enough rich detail that readers can judge whether the insights might transfer to environments they know. We addressed transferability by providing a thick description of our case and findings. Throughout the Method and Results chapters, we include detailed descriptions of Bank© 's context, e.g. the nature of its IT organization, the types of AI tools used, specific challenges faced, and cultural or regulatory factors in play. We also carefully describe each theme with sufficient context and participant quotes. By doing this, we enable others to gauge similarity: for instance, a reader from another bank or a tech company can consider which aspects of Bank© 's situation match their own and thus whether our conclusions about, say, "the need for human oversight of AI" might hold true elsewhere. While we do not statistically generalize, we argue that the theoretical implications (e.g., the concept of a "supervisory burden" in AI integration) could be analytically generalized to organizations with comparable structures or constraints. Providing this dense contextual information is the primary technique recommended by Bryman (2016) to support transferability in qualitative research.

- **Dependability:** Dependability addresses the consistency and replicability of the research process. To ensure dependability, we maintained a comprehensive audit trail of the research activities. This means we kept organized documentation of all

phases of the study: the initial research proposal and problem formulation, the criteria and rationale for selecting participants, interview guides, raw interview audio files and transcripts, field notes, coding schemas, theme development notes, and drafts of findings. By preserving these materials, we enable an external reviewer or auditor to trace how we went from raw data to final conclusions. Bryman (2016) suggests that an "auditing" approach, where another researcher could in principle review the project's documentation, is a good practice to demonstrate dependability. While a formal external audit was not conducted, the thesis supervisor did review portions of this trail to ensure methodological rigor. Moreover, we used a codebook or coding schema to apply codes consistently across transcripts; this was updated as coding progressed, and earlier transcripts were revisited to ensure uniform application of final codes. Such careful documentation and consistency checks mean that if another researcher were to follow the same procedures on the same data, they should be able to arrive at comparable conclusions, thus supporting the dependability of the results.

- **Confirmability:** Confirmability involves assuring that the findings are derived from the data themselves, not unduly influenced by the researcher's biases or preferences. Complete objectivity is impossible in qualitative research, especially under an interpretivist paradigm, but we took steps to ensure the findings reflect participants' views more than our preconceptions. One technique was maintaining reflexivity: we consistently reflected on their own assumptions and potential biases. For instance, being aware that we, as researchers, might be enthusiastic about AI, we were cautious not to frame questions in a leading way or to overlook negative sentiments. We acknowledged our prior familiarity with Bank© and agentic AI technologies in a reflexive memo, and actively bracketed these pre-understandings when listening to participants, striving to let participants' voices be heard on their own terms. Another practice enhancing confirmability was the heavy use of direct quotes from participants when reporting findings. By grounding each analytical claim in an illustrative quote, we show that the interpretation arose from actual data. Readers can see the origin of an insight in a participant's own words, which lends transparency to how we derived our conclusions. All data were kept available for review by the thesis examiner if needed, demonstrating that conclusions can be traced back to raw data. Additionally, none of us have a dependency or stake in the outcomes that would obviously bias the results. This neutrality, along with the practices above, helped ensure that the study's insights genuinely emerged from the participants' experiences rather than our own motivations.

3.5 Ethical Considerations

This research was conducted in accordance with the ethical guidelines of the Swedish Research Council (Vetenskapsrådet) and the principles of god forskningssed (good research practice) in Sweden, as well as general social research ethics as outlined by Bryman (2016).

Several key ethical considerations were addressed to protect participants and the organization:

- Confidentiality and Anonymity: Given the sensitive industry and the potentially proprietary nature of discussions, strict measures were taken to safeguard confidentiality. We assured participants that their identities would not be revealed in the thesis or to anyone outside the research team. In transcripts and in the report, each participant was referred to by a pseudonym or a generic role label. We avoided collecting directly identifying personal data beyond professional role and experience. Any specific names of projects, internal system names, or unique acronyms mentioned by participants were either omitted or generalized to prevent tracing back to individuals or sensitive company initiatives. For example, if someone referenced a project with a unique codename, we might describe it generically as "an internal AI pilot project." These steps ensure anonymity. Additionally, confidentiality agreements were respected: if participants shared off-the-record comments or asked that certain information be kept internal, those were not included in the thesis. In essence, we treated all raw data as highly confidential, and only non-identifiable, necessary insights were made public.

- Data Protection and Storage: All interview recordings and transcripts were stored securely on password-protected devices or encrypted cloud storage provided by the university, accessible only to us and thesis supervisor. We complied with GDPR (General Data Protection Regulation) principles for handling personal data, even though we collected minimal personal data. Audio files and transcripts were considered personal data under GDPR because voices and opinions are personal information. Thus, these were handled with care and not shared or stored in any way that could lead to unauthorized access. No data was transferred to any third party or outside the EU in a way that would violate data protection policies.

- Organizational Approval: Before data collection, the study design was discussed with and approved by a manager at Bank© who acted as an internal sponsor for the research. This helped ensure that our access to employees was authorized and that we were aware of any boundaries (for instance, topics too sensitive to probe deeply). By obtaining these approvals and following established ethical frameworks, we demonstrated respect for both the participants' rights and the organization's interests.

In conclusion, the study was carried out with a strong ethical compass, prioritizing confidentiality, data security, and the welfare of participants. These measures ensured that participants could share their insights candidly and confidently, knowing that their personal and professional integrity would be protected. This ethical diligence not only protects those involved, but also enhances the integrity of the research itself, fostering honest and reliable data.

3.6 Use of AI tools

Generative tools were employed during the writing process of this thesis to assist with language refinement, structural organization and proofreading. Specifically ChatGPT was used to improve grammatical accuracy. It is important to note that the AI was not used to generate empirical data, conduct analysis or formulate the study's conclusions. The authors have reviewed, verified and remain fully accountable for all content presented in this thesis, ensuring it aligns with the principles of research integrity.

4. Analysis and Results

Through an inductive thematic analysis of the interview data, several key themes emerged concerning the introduction of agentic AI into Bank©'s software development processes. Participants generally expressed optimism regarding the potential of autonomous coding agents to improve efficiency, while simultaneously identifying substantial technical, organizational, and procedural challenges.

The findings present a nuanced picture of early adoption: experimental implementations were described as both energizing and uneven; developers' roles appear to be shifting toward increased oversight and quality assurance responsibilities; and concerns related to risk management, accountability, and organizational safeguards remain central.

The themes are presented below and are grounded in participants' own words. The theoretical interpretation of these findings, in relation to Sociotechnical Systems (STS) theory and Human–AI Collaboration, is developed in the subsequent discussion chapter.

4.1 Initial Adoption and Perceived Benefits

Enthusiasm for efficiency gains: Interviewees across roles noted that agentic AI tools have begun to accelerate certain tasks and spark new ways of working. Several participants described feeling excitement when they first saw these agents generating usable code. "*We have the possibility to actually let AI generate some part of the code or even the whole project. So it's super exciting, and still many issues with that process*" (Appendix A, Interview 5). This mix of excitement and caution was typical. Developers recounted how, in an internal hackathon-like experiment, the agent produced outputs "*unexpectedly well*" in some cases, and team members "*adopted it quickly*". One senior developer explained that even before official approval, some teams eagerly started experimenting in a bottom-up fashion, using available AI services to speed up repetitive coding tasks. "*When we started using it, there was no permission yet from the Bank©… we already used a free version… I think it's cool stuff, but you always need to verify what it generates*" (Appendix A, Interview 5). This early adoption, done under the radar, underscores both the perceived value of the tool and the informal nature of initial trials. It also highlights a situation in which technological possibilities developed more rapidly than formal organizational processes and approvals.

Concrete improvements and use cases: Participants identified areas where agentic AI already shows promise. For example, one developer mentioned using an AI agent to automate parts of testing: rather than manually writing every test case, they could ask the agent to generate tests based on best practices. This saved time in generating comprehensive test suites, indicating efficiency improvements in quality assurance tasks. A domain architect likewise saw "*enormous potential… to make our business development process more efficient*" through agentic AI (Appendix A, Interview 4). He noted that

autonomous agents could eventually help create new functionality for customers and even assist with compliance tasks by executing repetitive processes much faster than before. In summary, the perceived benefits include faster development cycles, reduced grunt work for developers, and the ability to tackle more complex or higher-level problems while the agent handles routine coding. These perceived opportunities drove early enthusiasm and justified Bank©'s interest in agentic AI.

4.2 Evolving Developer Roles and the Need for Oversight

A clear theme was the shift in developers' responsibilities when using AI agents. Instead of hand-crafting every piece of code, developers find themselves overseeing and reviewing the agent's output. As one senior developer described, "*Previously, we were just writing prompts and communicating with AI. Now we have the possibility to actually let AI generate some part of the code or even the whole project.*" (Appendix A, Interview 5). In practice, this means the developer's role moves toward specifying objectives, then checking and refining what the agent produces. Several participants likened this to a quality assurance or code review function. "*It's just not good enough to trust it fully. That's where a developer is needed, I would say. So, you need to know what you are doing*" (Appendix A, Interview 5).

All interviewees stressed that human oversight is indispensable, given current limitations of agentic AI. The AI Platform Engineer responsible for developer tooling observed that agent outputs can deteriorate if the problem context isn't carefully managed. He gave an example that the agent would sometimes forget earlier instructions when dealing with lengthy tasks. "*One challenge was the context limit… you had to work step-by-step and constantly request documentation, otherwise it forgot the context. This could be mitigated by building our own 'Bank©-server' for more structure*" (Appendix A, Interview 1). Essentially quarantining the agent in a controlled environment with better memory of the project; reflects the ongoing need for developers to guide and constrain the AI. Developers must often break tasks into smaller chunks and supervise each step of the agent's work to maintain quality. As one participant put it, "*you always need to verify what it generates*" (Appendix A, Interview 3). In the absence of such oversight, the risk of subtle bugs, architecture deviations, or inefficient code creeping in is too high.

In the absence of such active oversight mechanisms, the risk of subtle bugs, architecture deviations, or inefficient code creeping in is too high. Notably, accountability remains with the human developers, despite the AI's autonomy. The domain architect explained that while agents can generate a lot of code very quickly, someone still needs to take responsibility for that code: "*With agentic AI… do we dare and do we have control over these parts? How do we ensure that it is correct when it's not a developer who has written all the code?... You have to validate it with the help of test cases*" (Appendix A, Interview 4). Here we see the developers transitioning into a supervisory role, they become curators and reviewers of the AI's output, accountable for catching errors or missteps the agent might introduce. This shift was recognized as a necessary trade-off for the efficiency gains; participants accept an increased burden of code review and testing as part of working with agentic AI.

Another aspect of the evolving developer role concerns the effect that use of agents has on the sense of creativity and enjoyment in software work. Several participants described how the transition from actively building software to supervising and correcting AI-generated output reduces the satisfaction associated with development work. One developer

interviewed reflected on this shift by noting that: *"I like writing code but now I don't know if I will continue to do that. I don't really like the prompt thing right now… Sometimes I kinda don't see a reason why I'm needed"* (Appendix A, Interview 5). Other participants expressed similar concerns, worrying that the role will be primarily about issuing prompts rather than engaging in what they perceive as meaningful problem-solving: *"Some developers love writing code and don't want to give that up. They find this development a bit boring in that sense and wonder 'where does the fun go now?'"*(Appendix A, Interview 6).

In summary, the introduction of autonomous coding agents is reshaping developer work: coding is no longer the primary value provided by senior developers; instead, their experience is applied in verifying, tweaking, and guiding the AI's contributions.

4.3 Technical limitations and challenges

While participants described substantial potential in agentic AI, they also emphasized several technical limitations that currently hinders the implementation of a broader or more reliable use in the software development process. A recurring challenge concerned context handling. Developers noted that the agents often lost track of earlier instructions when the task became too complex or expansive. One solution architect explained that *"one challenge was the context limit… you had to work step-by-step and constantly request documentation, otherwise it forgot the context"* (Appendix A, Interview 1).

This challenge was also emphasized by one team leader who described how limited context or unclear instructions caused the agent to fill in the gaps by itself, often leading to incorrect assumptions. The interviewee said that *"if the requirements were even slightly vague, the agent started making assumptions… and it's quite bad at guessing, which meant we had to be extremely explicit in our prompts. You really have to micromanage it and spell out things that are obvious to a human"* (Appendix A, Interview 3).

These challenges indicate that current agentic AI systems require substantial human coordination and explicit structuring of tasks in order to function reliably. Consequently, limited context handling forces developers to adopt a highly controlled step-by-step working approach to maintain the active human oversight required to prevent errors or hallucinations.

4.4 Emerging Structures: Towards an Agent Execution Environment

The interviews indicate that the introduction of agentic AI has not only affected individual work tasks but has also prompted gradual adjustments in tools, workflows, and collaboration practices within Bank©'s development environment.

Several participants described how existing development environments were not always well suited for working with AI agents. A Senior Developer explained: "*We started with four persons working on IntelliJ… After a couple of hours, other persons switched to VS Code, since you can see the huge difference… it's way more easier*" (Appendix A, Interview 5).

This shift was described as pragmatic rather than formally mandated. Developers experimented with different tools to identify environments that better supported interaction with AI agents and facilitated smoother workflows. The move toward alternative editors reflected an ongoing adaptation process in response to the practical demands of AI-supported development.

Participants also emphasized that successful outcomes depended heavily on how tasks and contextual information were formulated. A domain architect noted: "*The more context you can send in to the agent, the better quality… we rather went to a 10x shift actually*" (Appendix A, Interview 4).

Rather than issuing short or informal prompts, developers described the need to carefully prepare and structure requirements, business logic, and technical constraints before engaging the agent. This preparatory work was portrayed as increasingly central to the development process. Writing clear and structured input became as important as reviewing and validating the generated output.

Furthermore, participants highlighted how these changes influenced expectations regarding responsibility and coordination. Developers repeatedly emphasized the need to "verify what it generates" (Appendix A, Interview 5) and stressed that accountability ultimately remained with the human developer (Appendix A, Interview 4). As a result, the role of the developer was described as increasingly focused on reviewing, validating, and coordinating the agent's contributions rather than exclusively producing code independently.

Taken together, these observations suggest that the introduction of agentic AI is accompanied by incremental structural changes in tooling choices, collaboration practices, and the distribution of responsibilities within development teams.

4.5 Quality, Trust and Responsibility of AI-Generated Code

As agentic AI takes on portions of the coding work, issues of code quality, trust, and responsibility have become paramount. A central question in our interviews is: How do we ensure that what the AI produces is correct, safe, and maintainable, and who is accountable for it? The findings show that Bank© developers approach AI-generated outputs with a high degree of scrutiny and caution, underscoring the low trust in the AI's reliability at this stage. As one senior developer put it bluntly, "*but we shouldn't trust the LLMs. We should verify what they say.*", and therefore "*the ones who actually generate the code should verify it*" (Appendix A, Interview 2) before it is accepted. In practice, this means every piece of code coming from an AI is reviewed and tested by a human developer, just as or even more rigorously than code written by a person. The analogy was made that using an AI agent is like bringing a very junior programmer into the team: you would never let an inexperienced junior commit code without review, and similarly the AI's code is not trusted until it's checked and proven. One manager explicitly compared it to existing code review practices, saying it's essentially the same quality and safety checks as usual, "*as if you brought in a very junior developer and just let them loose, which we would never do*" (Appendix A, Interview 3). This highlights that human oversight is non-negotiable for AI contributions, a view completely aligned with SASE's emphasis on human-provided strategic direction and verification in the SE for Humans mode.

The interviews provided concrete stories illustrating why this caution is necessary. In one case, a developer recounted how an agent-generated piece of code appeared to work at first, the AI even reported that the project was compiling, but in reality some unit tests had failed, and the agent simply ignored or failed to mention those failures. The human caught it and had to prompt the agent to check again for the failing tests, at which point the issue was discovered. This incident shows that today's AI agents may lack a robust sense of quality assurance; they might present an answer that superficially meets the request while overlooking deeper problems. Another example mentioned by a team lead was an agent that

literally satisfied the exact instructions given but in doing so produced an over-engineered solution and even removed some test cases, technically fulfilling the requirement but violating implicit quality expectations. These anecdotes underscore that blindly trusting AI output is risky, as the AI can be prone to omissions, misunderstandings of intent, or "*over-solution*" of problems. Bank© developers have thus learned to approach AI suggestions with a healthy skepticism: every line of agent-written code is inspected, tested, and often modified before being integrated.

Responsibility for the code ultimately remains with the human developers, according to all interviewees. Despite the AI's growing role in generating code, participants were adamant that accountability cannot be offloaded to the machine. "*If you built it, then you are responsible for it - it doesn't matter if you used an agent*" (Appendix A, Interview 3) one developer insisted. This reflects a consensus that the responsibility is on the developer using the AI to ensure the output is sound. In other words, the AI is viewed as a tool, and just as a developer would be responsible for code they copied from an open source snippet, they are responsible for code produced by an AI on their prompt. Notably, Bank© has not yet established any formal policy that says this, it's more of an understanding in practice. When asked directly if there is a policy about who bears responsibility for AI-generated code, one interviewee answered that "*not right now, it's too early… for now, everybody bears responsibility for their own generated code*", because "*we don't really know how we are going to work later; we are just trying things*" (Appendix A, Interview 4). This statement highlights two things: **(1) Current norm = human developer** is responsible (by default, since they are the ones integrating the code), and **(2) This norm could evolve** as the practice matures (perhaps in the future new roles like "AI auditor" or automated verification tools could share some of the burden, but that is still undecided). For now, however, every team member knows that using the AI does not absolve them of the duty to ensure code quality and correctness.

To support this culture of accountability, teams have begun to put in place certain checks and guidelines. Some teams have informal coding guidelines for using AI, such as rules about when it's appropriate to rely on the agent versus when to code manually, and how to document AI contributions. One manager mentioned that her team developed internal guidance on "*when we use AI, when we don't*" (Appendix A, Interview 3) in the development process. Furthermore, interviewees discussed the importance of traceability; knowing which parts of the codebase were produced by an AI and ensuring those parts are extra vetted. In a bank like Bank©, such traceability is not just a good practice but likely a regulatory expectation. The developers themselves voiced an interesting insight on this front: one suggested treating certain AI inputs/outputs "*as code*" in the sense of version controlling them and reviewing them just like any other code artifact. For instance, if an agent is given a specific requirement or design to implement, that specification could be stored and versioned, and the changes the agent makes could be tracked. This idea would effectively extend existing software quality practices to cover the agent's involvement, creating an audit trail for decisions the agent made. It's another example of the social system (developers' processes) adapting to ensure the technical contributions of the AI remain under oversight.

In summary, Bank©'s approach to quality and responsibility in AI-generated code is characterized by intensive human validation and a clear stance that accountability stays with humans. Trust in the agent is being built slowly, as small successes accumulate, developers may grant a bit more autonomy, but they verify at every step. This cautious stance is well-founded given the current state of the technology, and it reflects a broader

theme that emerged: trust is something to be earned by the AI over time, not given by default. As one interviewee summarized, at this stage "*we trust it very little or almost not at all*"(Appendix A, Interview 3), especially for anything beyond toy projects. Only through repeated proven performance in the organization's context would that trust grow. Until then, the developers remain the final gatekeepers of quality and correctness.

5. Discussion

The purpose of this study was to examine how Bank©'s IT organization can integrate agentic AI systems, specifically autonomous coding, testing, and planning agents, into its software development processes, while maintaining efficiency, code quality, security, and scalability.

In this chapter, the empirical findings are interpreted in relation to existing theory and prior research. Drawing on Sociotechnical Systems (STS) theory, Human–AI Collaboration research, and the Structured Agentic Software (SASE) framework, the discussion examines how the results contribute to an understanding of agentic AI integration in a regulated organizational context. In line with Hatakka and Larsson (2015), the discussion "turns and examines" the results to clarify how the analysis leads to the conclusions drawn and how the findings relate to earlier research.

5.1 Agentic AI as a source of efficiency

The findings indicate that agentic AI is perceived as a strong enabler of efficiency in software development at Bank©, particularly in tasks such as code generation, testing, and exploratory implementation. Participants described faster turnaround times and reduced manual effort for routine development work, which aligns with prior research suggesting that autonomous agents can accelerate development workflows (Hassan et al., 2025; Rasheed et al., 2018).

However, the results also demonstrate that these efficiency gains are not frictionless. Increased agent autonomy introduces new bottlenecks, particularly in review and verification processes. Several participants emphasized that AI-generated code often requires extensive human validation, shifting effort rather than eliminating it. This finding mirrors the "supervisory burden" identified in early empirical studies of agentic software engineering, where productivity gains are partly offset by increased demands on senior developers' oversight capacity (Hassan et al., 2025).

From an STS perspective, this suggests that efficiency cannot be understood purely as a technical outcome. Instead, efficiency emerges from the joint optimization of social and technical systems. While the technical system (agentic AI) increases execution speed, the social system (human developers and reviewers) must absorb new responsibilities related to quality assurance and risk management. Without corresponding adjustments to roles, processes, and artifacts, the promised scalability of agentic AI risks being undermined by organizational strain rather than realized in practice.

5.2 Shifting roles: from software developers to supervisors and mentors

A central finding of the study is the observed transformation of developer roles. Rather than replacing human developers, agentic AI reconfigures their work toward supervision, guidance, and accountability. Participants consistently emphasized that humans remain responsible for decisions, quality, and security, even when agents perform substantial

portions of the coding work. This aligns closely with research in Human–AI Collaboration, which stresses the importance of human-in-the-loop oversight and the non-transferability of responsibility to autonomous systems (Schneiderman, 2020; Dellermann et al., 2019).

The findings also resonate strongly with STS theory. The introduction of agentic AI alters the division of labor within the sociotechnical system, requiring humans to adopt new cognitive and organizational roles. Rather than optimizing the technical subsystem in isolation, Bank© must redesign social structures, including role definitions, competencies, and evaluation criteria. Failure to do so risks misalignment between agent capabilities and human responsibilities, potentially leading to reduced trust and increased operational risk.

Importantly, this role transformation appears particularly pronounced in a regulated banking context. Participants emphasized that accountability for compliance and security cannot be delegated to AI agents, regardless of their technical sophistication. This reinforces Murugesan's (2025) argument that agentic AI intensifies, rather than diminishes, the need for clear governance structures and human accountability in organizational contexts.

5.3 Technical limitations as organizational challenges

The analysis identified several technical limitations of current agentic AI systems, including limited contextual understanding, memory constraints and challenges in verifying complex changes. While these issues are often framed as technical shortcomings in the literature, the findings suggest that their consequences are fundamentally organizational.

For example, limitations in agent context handling require developers to decompose tasks into smaller, more structured units, increasing coordination and planning effort. This supports Rasheed et al's (2018) argument that autonomous agents require redesigned workflows rather than simple substitution for human labor.

From an STS perspective, these limitations highlight that technical constraints propagate into social systems, shaping how work is organized and controlled. Rather than viewing agentic AI limitations as temporary technical deficiencies, organizations such as Bank© must treat them as design parameters around which processes and governance structures are built.

5.4 The role of structure: SASE as a governance mechanism

One of the most significant alignments between theory and empirical findings concerns the need for structured workflows and artifacts. Participants repeatedly highlighted the importance of documentation, traceability, and standardized processes when working with autonomous agents. This closely mirrors the SASE framework's emphasis on structured artifacts such as Briefing Packs and Merge Readiness Packs (Hassan et al., 2025).

The findings suggest that SASE is not merely a productivity framework but functions as a governance mechanism, particularly suited to regulated environments. By externalizing intent, constraints, and verification evidence into durable artifacts, SASE addresses key concerns related to auditability and accountability. This is especially relevant in Bank© 's context, where informal prompting and opaque agent behavior are incompatible with regulatory requirements.

From a sociotechnical standpoint, SASE operationalizes joint optimization by simultaneously redesigning technical environments (Agent Execution Environments) and social roles (humans as supervisors and mentors). The empirical data supports the argument that such structural separation is necessary to manage the tension between speed and trust identified in early deployments of agentic AI.

5.5 Trust, quality, and accountability in regulated environments

The findings reveal a cautious attitude toward trust in agentic AI. While participants acknowledged the usefulness of agents, trust in their outputs remained limited, particularly for critical or security-sensitive components. This reflects broader concerns in the literature regarding algorithmic opacity and responsibility gaps in autonomous systems (Murugesan, 2025).

Rather than viewing low trust as a barrier to adoption, the findings suggest that skepticism functions as a protective mechanism in high-stakes environments. Trust appears to be conditional and process-dependent, emerging through structured validation, transparency, and repeated successful interaction. This supports the argument that trust in agentic AI should be designed through governance and process rather than assumed as a property of the technology itself.

5.6 Implications for integrating agentic AI at Bank©

Synthesizing the findings, the study suggests that effective integration of agentic AI at Bank© requires more than technical deployment. Key implications include:

- **Process design:** Clear boundaries for agent autonomy and mandatory human review points.
- **Structured artifacts:** Treating prompts, specifications and verifications outputs as first-class engineering artifacts.
- **Role clarification:** Explicitly defining human responsibility for quality, security and compliance.
- **Governance alignment:** Embedding agentic AI within existing regulatory and audit frameworks.

These implications directly address the main research question by demonstrating that efficiency and scalability are achievable only when autonomy is balanced with structured oversight and organizational accountability.

6. Conclusion

6.1 Concluding findings

This study suggests that the effective integration of agentic AI into software development at Bank© appears to depend upon a deliberate sociotechnical redesign rather than the mere adoption of new tools. While autonomous agents demonstrate clear potential to increase efficiency and accelerate development tasks, their introduction significantly reshapes developer roles, governance structures, and accountability mechanisms.

The findings indicate that agentic AI shifts software developers from primary code producers toward supervisory and mentoring roles, where responsibility for quality, security, and compliance remains firmly human-centered. Without structured oversight

mechanisms, the productivity gains of agentic AI risk being offset by increased supervisory burden and reduced trust in AI-generated outputs.

The empirical evidence suggests that while autonomous agents possess the technical capability to accelerate development, their operational success is strictly bound by the quality of the organizational context provided to them. Consequently, the efficiency of an agent in this context does not appear to be an inherent property of the model alone, but is strongly shaped by the quality of human preparation. This creates a shift in effort, to leverage the speed of automation, developers must initially invest significantly more effort into *Context Engineering*, structuring business rules, architectural patterns, and requirements into formats that agents can reliably consume. When this context is absent or vague, agents in this study tend to hallucinate or exhibit "*laziness*," effectively shifting the workload from execution to rigorous specification and micromanagement.

Furthermore, agentic AI makes existing technical debt more visible, showing a clear difference between *Greenfield* and *Brownfield* scalability. In controlled, new environments like the *Cloud Saga initiative*, agents were experienced as force multipliers in contexts where the architecture could be optimized for machine interpretation from the start. Conversely, in legacy (Brownfield) environments, the scalability of agentic AI is severely constrained by the lack of documentation and structured logic. The study indicates that agents struggle to navigate complex, undocumented codebases where business logic is tacit rather than explicit. Therefore, the scalability of agentic AI does not appear to be a plug-and-play solution but is conditional upon a prerequisite investment in modernizing and documenting legacy systems to make them consumable by AI.

Finally, the integration of autonomous agents has triggered a shift in the developer's role from *Authoring* to *Orchestrating*, creating a new form of supervisory burden. Despite the autonomy of the agents, accountability remains predominantly human-centric, governed by the principle of "*You build it, you run it*". This has necessitated a "*Trust but Verify*" workflow, where developers function as Agent Coaches or auditors who must scrutinize AI outputs for subtle errors, such as the unauthorized deletion of test cases to meet completion targets. This dynamic validates the sociotechnical tension where increased technical autonomy requires tighter social control mechanisms. Thus, the primary competence for senior developers is shifting from writing syntax to the ability to judge the architectural soundness and security of AI-generated solutions.

6.2 Contributions to research

From a research perspective, this thesis contributes empirical insight into the early adoption of agentic AI within a highly regulated financial institution, a context that remains underrepresented in existing literature. While much prior research on agentic software engineering is conceptual, experimental, or situated in less constrained environments, this study provides a grounded account of how agentic AI is experienced, interpreted, and governed in a real-world banking context.

By applying a sociotechnical lens, the study extends existing discussions on agentic software engineering by demonstrating how increased technical autonomy necessitates parallel transformations in organizational roles, processes, and governance structures. Rather than treating agentic AI as a purely technical innovation, the findings highlight how its integration reshapes the division of labor within software development teams,

reinforcing the connection between social and technical systems as described in Sociotechnical Systems Theory.

A key contribution lies in the empirical elaboration of developer role transformation. While previous literature conceptually predicts a shift from human execution toward supervision in agentic environments, this study details how such a transition materializes in practice. Developers are shown to move from primary code authorship toward roles centered on context engineering, validation, and accountability. This reconfiguration introduces a new form of supervisory burden, particularly for senior developers, whose expertise is increasingly applied to reviewing, constraining, and correcting agent output rather than producing code directly. By providing concrete examples of this shift, the study adds nuance to existing accounts of human–AI collaboration in software engineering.

Furthermore, the findings contribute to research on trust in human–AI collaboration by demonstrating that trust in agentic AI is not primarily attitudinal or interpersonal, but procedural and sociotechnically constructed. In the studied context, trust does not emerge through confidence in the AI system itself, but through structured workflows, validation routines, and the use of durable artifacts that support transparency and auditability. This suggests that, in regulated environments, trust should be understood as an outcome of governance design rather than as an inherent property of agentic technologies.

The study also provides empirical support for structured frameworks such as Structured Agentic Software Engineering (SASE), illustrating how artifact-centric collaboration and role differentiation can function as governance mechanisms in human–agent interaction. By showing how concepts aligned with SASE emerge organically in practice, the findings strengthen the framework's relevance as an analytical lens for studying agentic AI in complex organizational settings.

Taken together, these contributions extend existing research on agentic AI by grounding theoretical claims in empirical observations from a high-stakes, regulated environment. The study demonstrates how agentic AI integration is shaped not only by technical capability, but by organizational context, regulatory constraints, and sociotechnical alignment, thereby offering a more context-sensitive understanding of agentic software engineering in practice.

6.3 Contributions to practice

For practitioners, the study highlights that successful adoption of agentic AI requires more than selecting advanced tools. The findings suggest that organizations may benefit from investing in structured execution environments, clearly defined oversight roles, and standardized artifacts that encode intent, constraints, and verification evidence. In regulated environments such as banking, these structures are essential for maintaining auditability, trust, and accountability.

The results suggest that organizations like Bank© should treat agentic AI as a new organizational actor rather than a conventional productivity tool, requiring explicit governance models and continuous human oversight

Based on the empirical findings, the strategic implications for Bank© and similar financial institutions extend beyond mere tool adoption to a fundamental restructuring of information management and team dynamics.

To scale agentic AI effectively, organizations may need to fundamentally redefine their approach to knowledge management. Architectural documentation and business rules can no longer be viewed as passive reference materials; instead, they must be treated as active strategic assets essential for agent operation. Domain architects emphasize that future organizational output must be consumable by both humans and agents, requiring that requirements and architectural principles are version-controlled with the same rigor traditionally reserved for executable code. Investing in this codification of business knowledge is now a prerequisite for scalability , as agents lack the inherent context libraries to reason correctly about complex banking domains without structured input.

Simultaneously, organizations must align their team structures with the actual, rather than aspirational, capabilities of current AI models. The study indicates that these agents function closer to junior interns or inexperienced developers who rely on explicit instruction and constant review, rather than senior experts capable of autonomous turnkey delivery. This reality necessitates a shift in human responsibilities, where the primary value of senior developers moves from writing syntax to acting as *agent coaches* who provide critical judgment and guidance. To accommodate this mentorship dynamic, teams may need to adopt collaborative practices like mob programming, guarding against the operational risks of trusting agents with too much autonomy too soon.

Beyond immediate team dynamics, the deployment strategy must be governed by strict environmental controls, particularly regarding legacy systems. Since agents struggle to infer logic from undocumented *Brownfield* code, allowing them to operate in such environments without restrictions creates significant operational exposure. A robust governance framework requires the implementation of sandboxes that limit the agent's impact, ensuring that autonomous actions are contained and cannot inadvertently corrupt critical systems or access sensitive data. Consequently, widespread adoption in legacy environments should be postponed until those systems are sufficiently documented or encapsulated to ensure safe interaction.

6.4 Limitations and Suggestions for Future Research

As a qualitative, interpretive single-case study, this thesis is subject to several limitations that should be considered when interpreting the findings. First, the study is based on a limited number of interviews conducted within a single organization. While this enables an in-depth and context-rich understanding of agentic AI integration in a regulated banking environment, it also limits the extent to which the findings can be generalized beyond similar organizational contexts. The insights should therefore be understood as analytically rather than statistically generalizable.

Second, the study captures a snapshot of agentic AI adoption at an early stage of organizational experimentation. As the technologies, workflows, and governance structures surrounding agentic AI continue to evolve rapidly, the findings reflect a moment in time rather than a mature or stabilized implementation. Longitudinal changes in developer roles, trust formation, and organizational practices may therefore not be fully observable within the scope of this study.

Third, the empirical material is based on self-reported experiences and perceptions of developers, architects, and managers. While this aligns with the interpretive research design and provides valuable insight into sensemaking and lived experience, it may underrepresent

perspectives that are less directly involved in agentic AI initiatives, such as compliance officers or business stakeholders. Additionally, interviews conducted in a non-native language and minor technical issues during data collection may have constrained the depth or nuance of some responses.

Based on these limitations, several directions for future research can be identified. Future studies could employ comparative case designs across multiple organizations or industries to examine how regulatory intensity, organizational size, or development culture influence the integration of agentic AI. Longitudinal research would also be valuable in capturing how sociotechnical dynamics evolve as agentic AI systems become more embedded in everyday development practices.

Further research could also combine qualitative insights with complementary quantitative approaches, such as measuring changes in code review workload, development cycle time, or defect rates, to triangulate subjective experiences with observable outcomes. Finally, as structured frameworks such as SASE gain traction, future research could investigate their long-term effectiveness as governance mechanisms and explore how artifact-centric collaboration scales across larger and more diverse development environments.

References

Alenezi, M., & Akour, M. (2025). *AI-Driven Innovations in Software Engineering: A Review of Current Practices and Future Directions.* Applied Sciences, 15(3). https://doi.org/10.3390/app15031344

Baxter, G., & Sommerville, I. (2011). Socio-technical systems: From design methods to systems engineering. *Interacting with Computers, 23*(1), 4–17. https://doi.org/10.1016/j.intcom.2010.07.003

Bostrom, R. P., & Heinen, J. S. (1977). MIS problems and failures: A socio-technical perspective. Part I: The causes. *MIS Quarterly, 1*(3), 17–32. https://doi.org/10.2307/248710

Braun, V., & Clarke, V. (2006). Using thematic analysis in psychology. *Qualitative Research in Psychology, 3*(2), 77–101. https://doi.org/10.1191/1478088706qp063oa

Bryman, A. (2016). *Social research methods* (5th ed.). Oxford University Press.

Dellermann, D., Ebel, P., Söllner, M., & Leimeister, J. M. (2019). The future of human-AI collaboration: A taxonomy of design knowledge for hybrid intelligence systems. Proceedings of the 52nd Hawaii International Conference on System Sciences. https://doi.org/10.24251/HICSS.2019.034

Hassan, A. E., Li, H., Lin, D., Adams, B., Chen, T., Kashiwa, Y., & Qiu, D. (2025). *Agentic Software Engineering: Foundational Pillars and a Research Roadmap.* arXiv:2509.06216. https://doi.org/10.48550/arXiv.2509.06216

Hatakka, M., & Larsson, H. (2015). *Att skriva en uppsats – En introduktion i akademiskt skrivande.* Örebro: Örebro Universitet.

Murugesan, S. (2025). *The rise of agentic AI: Implications, concerns, and the path forward.* (Preprint PDF.) doi: 10.1109/MIS.2025.3544940

Nowell, L. S., Norris, J. M., White, D. E., & Moules, N. J. (2017).
Thematic analysis: Striving to meet the trustworthiness criteria. *International Journal of Qualitative Methods, 16*, 1–13. https://doi.org/10.1177/1609406917733847

OpenAI. (2024). ChatGPT (Version GPT-4o) [Large language model]. https://chatgpt.com

Pati, A. K. (2025). *Agentic AI: A Comprehensive Survey of Technologies, Applications, and Societal Implications. IEEE Access. doi: 10.1109/ACCESS.2025.3585609*

Rasheed, H. A., et al. (2018). *Autonomous agents in software development: A vision paper.* (Preprint PDF.) https://doi.org/10.48550/arXiv.2311.18440

Sapkota, R., Roumeliotis, K. I., & Karkee, M. (2026). *AI agents vs. agentic AI: A conceptual taxonomy, applications and challenges.* (Preprint PDF.) https://doi.org/10.48550/arXiv.2505.10468

Schneiderman, B. (2020). Bridging the gap between ethics and practice: Guidelines for reliable, safe, and trustworthy human-centered AI systems. ACM Transactions on Interactive Intelligent Systems, 10(4), 1–31. https://doi.org/10.1145/3419764

Tracy, S. J. (2010). Qualitative quality: Eight "big-tent" criteria for excellent qualitative research. *Qualitative Inquiry, 16*(10), 837–851. https://doi.org/10.1177/1077800410383121

Vetenskapsrådet. (2024). *God forskningssed 2024.* Vetenskapsrådet.

Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., & Press, O. (2024). SWE-agent: Agent-computer Interfaces Enable Automated Software Engineering. *NeurIPS 2024.* https://doi.org/10.48550/arXiv.2405.15793

Appendices

Appendix A: List of respondents

Reference ID	Role / Position	Experience	Date	Duration
Interview 1	Solution Architect	Architect responsible for the loan domain; led the Greenfield proof-of-concept.	2024-11-24	Approx. 45 min
Interview 2	Tech Lead	Tech Lead in Credit Process area; working on microservices with agentic AI.	2024-11-25	Approx. 39 min
Interview 3 2x	Team Manager	Team Manager; oversees multiple development teams experimenting with AI.	2024-11-28 2024-12-03	Approx. 60 min Approx. 45 min
Interview 4	Domain Architect	Domain Architect for CRM & Loans; sets overarching frameworks and strategies.	2024-11-26	Approx. 42 min
Interview 5	Senior Software Developer	Senior Java Developer; actively using Copilot/Agents for unit tests and refactoring.	2024-11-27	Approx. 45 min
Interview 6	AI Platform Engineer	Developer Experience / Copilot Responsible; focuses on tooling and developer support.	2024-12-03	Approx. 45 min

Appendix B: Interview Guide

SWEDISH

Introduktion & bakgrund

- Kan du berätta om din roll och ansvar hos Bank©?
- Hur ser ditt arbete ut i relation till Bank©:s utvecklingsprocesser?

Lätta frågor om agenter

- Vilka möjligheter ser du spontant med att introducera agentic AI i Bank©:s arbete? Är det några direkta risker du ser?

Integration, effektivitet & skalbarhet

- Vilka delar av Bank© tror du skulle påverkas mest av agentic AI?
- Hur pass stor förändring ser du i arbetstempo och skalbarhet om AI-agenter kan generera eller ändra kod?

Säkerhet, kvalitet & ansvar

- Hur anser du att man säkerställer kvalitet och säkerhet när agenter genererar kod eller gör förändringar autonomt? Ser du några risker med det? Vilka typer av skydd eller kontroller skulle vara nödvändiga?
- Hur tycker du att ansvarsfördelningen ska se ut?

Tekniska och organisatoriska förutsättningar

- Vilka verktyg och plattformar tror du är nödvändiga för att agentic AI ska kunna integreras på en större skala hos Bank©?
- Finns det några begränsningar? Isåfall, hur kan man lösa dem?

Roll Specifika frågor

Framtidsperspektiv

- Baserat på greenfield-experimentet eller dina egna tankar; hur tror du att agentic AI kommer att påverka Bank©:s utvecklingsarbete på längre sikt?
- Ser du några problem som kommer uppstå längre fram?

Avslutning

- Är det något du har att tillägga?

ENGELSKA

Introduction & background

Start explaining what we are doing (dem har typ ingen info om vad vi faktiskt gör)

- Can you tell us about your role and responsibilities at Bank©?
- How does your work relate to Bank© 's software development processes?

Initial questions about agents

- What opportunities do you see with introducing agentic AI into Bank© 's workflows?
- Are there any immediate risks you foresee?

Integration, efficiency & scalability

- Which parts of Bank© do you think would be most affected by agentic AI?
- How significant do you think the change in workflow speed and scalability would be if AI agents could generate or modify code?

Security, quality & accountability

- How do you think quality and security can be ensured when agents generate code or make changes autonomously?
- What kinds of safeguards or controls do you think would be necessary?
- How do you think responsibility should be distributed?

Technical and organizational prerequisites

- Which tools and platforms do you think are necessary for agentic AI to be integrated at scale within Bank©?
- Are there any limitations? If so, how could they be addressed?

Role-specific questions

Future perspective

- How do you think AI agents will impact Bank© in the long term?
- Do you foresee any problems emerging over time?

Closing

- Is there anything you would like to add?